

考虑预测更新与运行平稳性的微网购电调控策略

摘要

对于问题一,

对于问题二,

对于问题三,

对于问题四,

最后,

关键词： 关键词； 关键词； 关键词； 关键词； 关键词

一、 问题背景与问题重述

1.1 问题背景

分布式光伏为社区提供了就地获取电能的途径，但其电力供给的时间分布与居民用电需求并不天然一致：发电充裕时可能出现剩余电量，而负载集中或光伏回落时又需要依赖外部电网。配置储能电池可以将不同时段电能供需连接起来，却也引入了充放电损耗、功率上限和安全储电范围等限制。因此，微网运营的关键并非单纯提高某一时段的自给率，而是在持续满足负载的前提下，协调购电时机、储能状态与预测风险，控制整个运行周期的购电支出。现实调度还存在“计划先制定、信息后到达”的矛盾。购电计划偏乐观可能引发高价紧急购电，过度保守则可能为未被利用的电量支付费用；中途调整购电安排有助于修正计划，但调整本身也有代价。因此，设计一个“预测—计划—修正—执行—结算”一体化的调度分析模型，对建设兼具经济性与可靠性的微网发电系统具有重要意义。

1.2 问题重述

本文以光伏电源、社区负载、储能电池及外网组成的系统为对象，将原题四问归纳为以下递进任务。

问题一要求利用典型日电价、负载及给定光伏预测，确定各时段的购电与充放电安排。我们建立两层混合整数线性规划模型，在日初日末储电量一致的条件下，先求最低购电费，再在小幅费用让步范围内改善电池充放电的平稳性。购电量作为题目要求的结果进行统计，不再单独最小化。

问题二要求先依据历史负载和光伏数据构建一个预测模型，根据预测得到的每日负载和发电数据制定每日 0 时计划。新的购电计划需要考虑如果计划供电低于实际负载，需要按当时电价 5 倍计收的紧急购电费用。

问题三引入每日 0、6、12、18 时发布的未来 24 小时整点光伏预报，研究是否以及如何修正未执行计划。重点衡量提前量误差、未来更新机会与上调 1.5 倍、下调 50% 相关计费之间的关系，并评价额外预报的实际决策价值。

问题四将固定日内价格扩展为随日期和时段变化的电价，在明确价格可见性的前提下分别重新研究无额外光伏预报和有日内预报的两类策略，比较价格预测误差与供需误差共同作用下的购电安排。

二、问题分析

四问共享能量守恒、储能状态递推与充放电硬互斥约束，差别在于决策时可用的信息及购电结算机制。本文按照“物理约束统一、历史信息预测、风险驱动决策、因果执行检验”的思路，逐层增加供需误差、日内预报和电价波动。数据对齐、参数拟合和误差校准均限定于当时已知信息，以保证各策略在同一可行域和可比的信息条件下评价。

问题 1 分析

本问的电价、负载和光伏预测已知，关键在于利用储能连接不同时段的供需，同时兼顾购电费用与充放电平稳性。首先按十分钟区间建立能量平衡和储电递推关系，以二元变量禁止同时充放电，并要求日初日末电量相等。随后结合功率变化和最短运行时间限制，先求当前运行条件下的最低电费，再在费用预算内减少功率波动与短动作。最后按指定时段汇总结果，通过经济基准对比、约束检查和参数扰动，分析平稳运行的代价及计划对预测偏差的适用范围。

问题 2 分析

关键矛盾是日前计划必须先于实际供需制定，而预测缺口与剩余购电的经济后果不对称。先用日、周及低自由度年周期动态回归刻画负载，以“季节日内背景乘短期发电状态”预测光伏；再利用历史相关误差估计连续净负荷缺口，将其转化为可选的自适应储能备用。备用参数通过过去的因果执行费用校准，而非只按预测精度选择。随后逐日完成预测、计划、实时执行和状态更新，保持跨日储电量连续，并比较无备用、固定备用与自适应备用策略。

问题 3 分析

关键矛盾是新预报能降低部分不确定性，但调整有成本，远期风险还可能被后续更新修正。首先按发布时刻与提前量匹配预报和实际值，分离偏差与方差，并控制昼夜差异；再生成保留时间相关性的误差路径，将整点功率校准后插值积分至 10 分钟。进而建立提前量感知的多阶段风险调度，使 0 时计划预先考虑未来更新机会，同一已知历史对应相同动作。预报到达后重新优化未执行区间，统一结算并通过不同更新时刻组合评价信息价值。

问题 4 分析

关键矛盾由供需预测误差扩展为供需与价格的联合不确定性，预知真实全日电价会夸大策略能力。首先以周期、近期及前周同刻价格建立滚动动态回归，随后分别接入问题 2 和问题 3，保持两类策略的光伏预报权限不变。若联合场景导致计算负担过大，再按决策成本压缩场景并回填遗漏的极端路径。最后在相同滚动时序下比较预测、费用与计算开销，区分价格环境变化、信息增加及算法增强各自的影响，全程不提前使用未来实测值。

三、模型假设

各问共同采用以下五项假设，问题一的典型日条件在后面补充说明。容量、功率、安全储电范围和供电要求属于题面约束，不作为可以放宽的假设重复列出。

假设 1：采用集中式单母线、区间平均功率表征。将微网视为统一调度的聚合系统，不另行刻画内部线路潮流、电压与频率动态；10 分钟控制区间内用平均功率描述能量交换。**合理性依据：**附件提供的是聚合功率与电价，没有内部拓扑、线路阻抗及秒级控制数据，不足以识别网络与暂态模型。**模型影响：**可用区间能量平衡连接负载、光伏、储能和购电；结论限于能量管理层，不声称验证配电网电气安全。整点预报通过插值后积分转换，不直接视为小时平均值。

假设 2：总程效率为 90%，且充、放电方向对称。取 $\eta_c = \eta_d = \sqrt{0.9}$ ，主模型中不另计自放电、辅助设备耗电与容量随时间衰减。**合理性依据：**题面只给出一个效率数值，没有分方向效率曲线及老化、辅助用电参数；对称分解是保持总程效率并减少不可识别参数的一种约定，而非由题面唯一推出的物理事实。**模型影响：**得到统一、可核验的储电量递推关系；通过效率及附加损耗情景检验结论敏感性，不将平稳性直接折算为延寿收益。

假设 3：负载不可主动削减，外网正常可用且不设置额外购电上限。所有负载均需得到满足；不设置售电收益，剩余电量允许不被利用，已申报购电仍按适用规则结算。**合理性依据：**题目要求保障供电，给出了紧急购电机制，却未给出可转移负荷、外网容量上限、停电过程及售电价格。**模型影响：**紧急购电承担最后的供需平衡补救，比较聚焦于费用而非失负荷；总剩余应区分光伏弃电与已付费未使用购电。本结论不适用于限电或孤岛场景。

假设 4：信息按时可用，决策与执行遵守因果顺序。已发布预报和已实现观测可在下一次允许决策时使用，不考虑超过 10 分钟的通信或执行延迟；问题 4 主情景暂按真实价格随时间揭示处理。**合理性依据：**题面明确了预报发布时间，但未提供通信故障和价格提前公告机制；逐步揭示价格是避免引入额外先知信息的工作解释。**模型影响：**只用过去信息拟合预测与风险参数，冻结已执行动作并传递实际储电量；价格全日已知仅作辅助情景，延迟影响可通过滞后一期的信息检验评估。

假设 5：经周期与近期状态调整后，历史误差在短期内具有局部可迁移性。不要求全年同分布、误差独立、无偏或服从正态分布，只认为相近时段及条件下的近期误差能提供下一阶段风险参考。**合理性依据：**日历周期可作为建模结构，但其强度与误差分布需随历史更新；在缺乏未来气象等信息时，局部历史是可获取的校准依据。**模型影响：**支持滚动回归、备用分位与相关场景生成；采用近期加权、分组收缩、分块重采样和滚动外推检验，发现漂移时更新窗口，避免将该假设当成已验证事实。

问题一的补充假设

补充假设 1：假设内容：给定光伏预测作为本问的确定性供给输入，外网可按计划提

供所需电量，不额外设置购电容量上限。**设立依据：**本问要求利用已给定预测制定计划，未给出预测误差分布或外网接入功率限制。**对模型的影响：**约束保证给定预测下逐段供电；预测偏离时的适用边界另作压力测试，不据此假定实际发电无误差。

补充假设 2： **假设内容：**将给定曲线视为可重复的典型日，以 6000 kWh 为日初、日末储电量，并循环连接午夜两侧的运行模式。**设立依据：**问题一给出相同的日负载、电价，要求日初日末储电量相等，附录给出初始电量 6000 kWh。**对模型的影响：**排除透支初始储能获得虚假节费的方案；循环运行条件避免把功率跳变和短动作转移到日界处。

其中，典型日循环是本问的运行约定，后续随机变化条件下仍需按真实储电量跨日传递。爬坡限值及最短运行时间作为平稳运行方案的可调设置，在模型中单独列出，不视为题面已给定的硬件参数。

四、符号说明

下表列出贯穿各问的公共符号，局部回归系数和辅助线性化变量在相应模型中定义。上标“ $\wedge$ ”表示预测量；日下标 d 在不致混淆时省略。 E 表示储电量， e 表示紧急购电量，两者严格区分。

表 1 主要符号及单位

符号	含义	单位
d, t	日期索引、日内调度区间索引, $t = 1, \dots, 144$	—
Δt	单个调度区间长度, 取 1/6	h
τ, h	预报发布时刻、距离发布时刻的预测提前量	h
$L_{d,t}, V_{d,t}$	负载、可用光伏的区间平均功率	kW
$\hat{L}_{d,t}, \hat{V}_{d,t}$	决策时可获取的负载、光伏预测功率	kW
$p_{d,t}, \hat{p}_{d,t}$	基础电价及其预测值, 交易倍数另计	元/kWh
$g_{d,t}$	每日 0 时申报的计划购电量	kWh
$q_{d,t}$	当前决策版本中调整后的总购电量	kWh
$e_{d,t}$	实际执行时为补足缺口购买的紧急电量	kWh
$a_{d,t}^+, a_{d,t}^-$	相对原计划的净上调量、净下调量	kWh
$P_{d,t}^c, P_{d,t}^d$	交流侧充电功率、放电功率	kW
$z_{d,t}^c, z_{d,t}^d$	充电、放电模式指示变量, 二者不可同时为 1	—
$E_{d,t}$	区间 t 开始时的储电量, $E_{d,145}$ 为日末值	kWh
$E_{\min}, E_{\max}$	安全储电量下限 1200、上限 10800	kWh
$P_{\max}$	最大充电或放电功率, 取 5000	kW
η_c, η_d	充电、放电效率, 均取 $\sqrt{0.9}$	—
$w_{d,t}$	供需平衡中的总未利用电量, 并非全部弃光	kWh
$R_{d,t}$	为应对未来缺口预留的电池内部能量	kWh
$\varepsilon_{\tau, h}$	对应发布时刻与提前量的光伏功率预报误差	kW
$b_{\tau, h}, \sigma_{\tau, h}$	该组误差的偏差、去偏后的标准差	kW
$\mathcal{I}_{\tau}$	决策时刻 τ 已可获取的信息集合	—
s, π_s	随机场景索引及相应概率	—
C_{tot}	计划、调整及紧急购电构成的总费用	元
α, λ	尾部风险置信水平、风险厌恶权重	—

其中, $a_{d,t}^+ = (q_{d,t} - g_{d,t})^+$, $a_{d,t}^- = (g_{d,t} - q_{d,t})^+$ 仅表示相对原计划的净偏差, 不预先确定多次调整的计费基准。预报误差取“实际值减预测值”, 其 MSE 单位为 kW^2 ; $\sigma^2 = \text{MSE} - b^2$ 须在同一分组与一致矩估计口径下使用。模型中的功率需乘以 Δt 才成为区间电量。

五、问题一的模型建立与求解

5.1 问题一模型的建立

问题一要求我们根据给定的负载、电价和光伏预测，制定当天的购电计划。储能电池能够把低价时段及光伏富余时段的电量留到后面使用，但一次完整充放电过程存在10%的能量损耗，容量和功率也有限制。因此，我们建立两层混合整数线性规划模型，在满足全天供需平衡的基础上，先控制购电费用，再改善充放电的平稳性。

将一天划分为 $T = 144$ 个区间，区间长度 $\Delta t = 1/6 \text{ h}$ 。附件中的时间按区间右端解释，0:10 对应 $[0:00, 0:10)$ ，0:00 + 1 对应 $[23:50, 24:00)$ 。微网系统存在以下供需关系：

$$g_t + V_t \Delta t + P_t^d \Delta t = L_t \Delta t + P_t^c \Delta t + w_t, \quad g_t, w_t \geq 0. \quad (1)$$

其中， g_t 为区间 t 向外网申报的购电量， w_t 为未被负载或充电利用的剩余电量，单位均为 kWh； L_t 为给定负载功率， V_t 为光伏预测功率， P_t^c 与 P_t^d 分别为交流侧充、放电功率，单位均为 kW。

$$E_{t+1} - E_t = \eta_c P_t^c \Delta t - \frac{P_t^d \Delta t}{\eta_d}, \quad \eta_c = \eta_d = \sqrt{0.9}. \quad (2)$$

其中， E_t 与 E_{t+1} 分别为区间 t 开始和结束时电池内部的储电量，单位为 kWh； η_c 与 η_d 分别为充电效率和放电效率，两者乘积 $\eta_{rt} = 0.9$ 为往返效率。储电量及日界条件为

$$E_{\min} \leq E_t \leq E_{\max} \quad (t = 1, \dots, T+1), \quad E_1 = E_{T+1} = 6000. \quad (3)$$

其中， $E_{\min} = 1200 \text{ kWh}$ 、 $E_{\max} = 10800 \text{ kWh}$ 为安全储电量下限和上限； E_1 与 E_{T+1} 分别为 0:00 和 24:00 的储电量。

同一区间内，电池不能同时充电和放电。引入模式变量，写为

$$z_t^c, z_t^d \in \{0, 1\}, \quad z_t^c + z_t^d \leq 1, \quad P_{\min} z_t^m \leq P_t^m \leq P_{\max} z_t^m, \quad m \in \{c, d\}. \quad (4)$$

其中， $m = c, d$ 分别表示充电和放电； z_t^m 为相应模式是否开启的指示变量，取 1 表示开启，取 0 表示关闭。 $P_{\max} = 5000 \text{ kW}$ 为最大单向功率， $P_{\min} = 1 \text{ kW}$ 为有效运行的计数判据，避免零功率状态被计为持续运行。由于存在二元变量，本文模型属于混合整数线性规划。

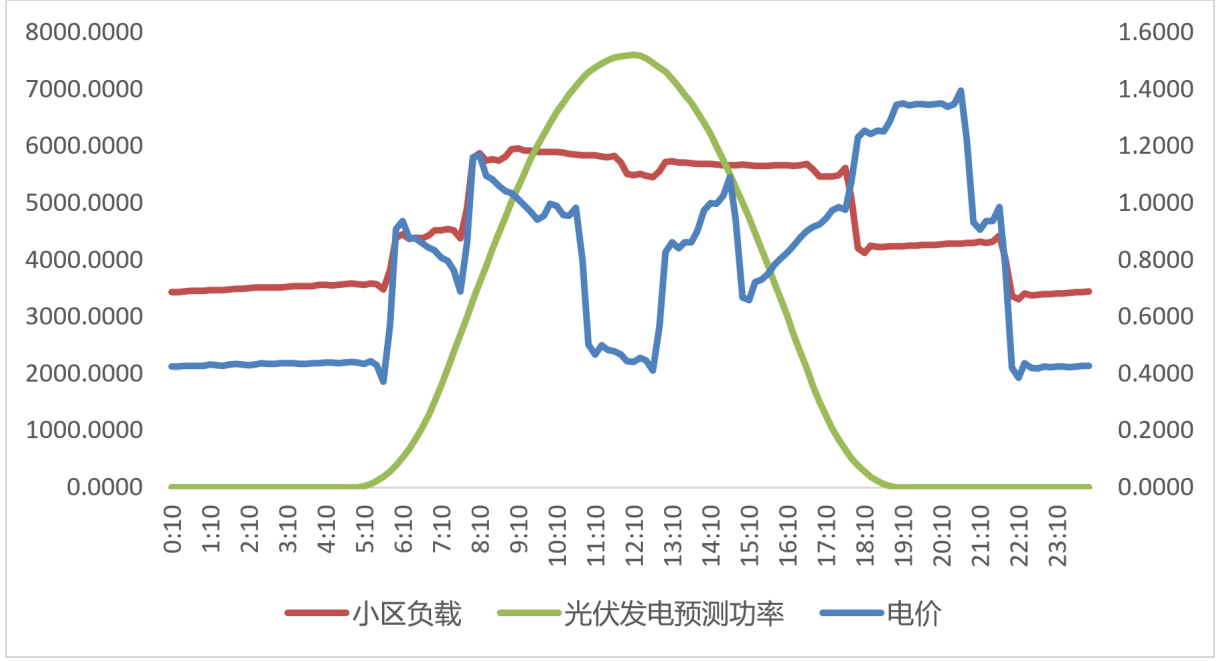


图 1 典型日负载、发电与电价

如图 1 所示，电价在相邻时段存在小幅变化。只追求最低电费时，模型可能把充电集中到少数更便宜的区间，使充放电功率频繁变化。考虑到小区储能设备需要持续运行，我们希望在少量增加费用的情况下，减少这些功率波动和短时启停。本文用运行平稳性衡量这一改善，不将其直接换算为电池寿命或折旧费用。

先定义净充电功率，并连接典型日午夜两侧的状态：

$$P_t^{\text{bat}} = P_t^c - P_t^d, \quad P_0^{\text{bat}} = P_T^{\text{bat}}, \quad z_0^m = z_T^m. \quad (5)$$

其中， P_t^{bat} 为净充电功率，单位为 kW，正值表示充电，负值表示放电；下标 0 用于表示前一循环日的末段。当天最后一段与下一天第一段同样需要满足运行要求。

$$-R_{\text{ramp}} \leq P_t^{\text{bat}} - P_{t-1}^{\text{bat}} \leq R_{\text{ramp}}. \quad (6)$$

其中， $R_{\text{ramp}} = 1000 \text{ kW}$ 为相邻十分钟平均功率指令的变化上限。这是本文选取的平稳运行设置，不是题面给定的设备参数。

为统计启停并限制短动作，建立模式转换关系：

$$\begin{aligned} s_t^m &\geq z_t^m - z_{t-1}^m, \quad s_t^m \leq z_t^m, \quad s_t^m \leq 1 - z_{t-1}^m, \\ o_t^m &\geq z_{t-1}^m - z_t^m, \quad o_t^m \leq z_{t-1}^m, \quad o_t^m \leq 1 - z_t^m, \end{aligned} \quad 0 \leq s_t^m, o_t^m \leq 1. \quad (7)$$

其中， s_t^m 表示模式 m 在区间 t 开始时是否启动， o_t^m 表示是否关闭，发生时为 1，否则为 0。两者由前后两段的二元模式状态确定，不是可以任意取零的启动、关闭计数。

$$\sum_{j=0}^{U-1} z_{t+j}^m \geq U s_t^m, \quad \sum_{j=0}^{D-1} (1 - z_{t+j}^m) \geq D o_t^m. \quad (8)$$

其中, $U = 3$ 、 $D = 2$ 分别为同一模式最短运行和关闭段数, 对应 30 分钟和 20 分钟; j 为相对区间 t 的偏移, 下标按典型日循环处理。关闭时间针对同一模式, 不表示整台电池必须停机 20 分钟。两项时长同样属于本文的运行设置。

第一层: 确定运行条件下的最低购电费。上述供需、储电、互斥及运行约束构成可行域 $\mathcal{X}$ 。两层求解采用相同的运行条件, 第一层先求最低购电费用:

$$F_1(x) = \sum_{t=1}^T p_t g_t, \quad C^* = \min_{x \in \mathcal{X}} F_1(x). \quad (9)$$

其中, x 表示由购电量、充放电功率、储电量、模式及辅助变量组成的完整调度方案, $\mathcal{X}$ 为满足上述全部约束的方案集合; p_t 为区间 t 的已知购电单价, 单位为元/kWh; $F_1(x)$ 为方案 x 的全天购电费用, C^* 为同一可行域内第一层的最优费用, 单位均为元。

第二层: 在费用预算内减少功率波动与短动作。用循环总变差 $\mathcal{V}$ 、启动次数 N_s 和短段缺额 B 衡量运行平稳性, 定义如下:

$$\mathcal{V} = \sum_t |P_t^{\text{bat}} - P_{t-1}^{\text{bat}}|, \quad N_s = \sum_t (s_t^c + s_t^d), \quad B = \sum_j (K - \ell_j^{\text{run}})^+, \quad (10)$$

其中, $\mathcal{V}$ 为含午夜边界的净功率总变差, 单位为 kW; N_s 为全天充、放电模式启动总次数; B 为短运行段统计量, 用于估计电池是否存在普遍的碎片化充放电行为, 单位为十分钟段。此式的 j 为循环运行段编号, ℓ_j^{run} 为第 j 个连续同模式运行段的长度, $K = 3$ 为短段参考长度; $(a)^+ = \max(a, 0)$ 表示取正部。

为保持线性结构, 引入辅助变量 ξ_t 表示绝对值的上界:

$$\xi_t \geq P_t^{\text{bat}} - P_{t-1}^{\text{bat}}, \quad \xi_t \geq P_{t-1}^{\text{bat}} - P_t^{\text{bat}}, \quad \xi_t \geq 0. \quad (11)$$

其中, ξ_t 为区间 t 与上一段净功率差绝对值的非负辅助上界, 单位为 kW; 两条下界分别覆盖功率差为正和为负的情况, 以线性不等式表达绝对值。在第二层的正权重最小化下, $\sum_t \xi_t = \mathcal{V}$ 。短段项则由启动后是否发生中断表达。对 $k = 1, \dots, K - 1$, 有

$$0 \leq b_{t,k}^m \leq s_t^m, \quad b_{t,k}^m \geq s_t^m - z_{t+j}^m \quad (j = 1, \dots, k), \quad (12)$$

$$b_{t,k}^m \leq k - \sum_{j=1}^k z_{t+j}^m, \quad B = \sum_{m,t,k} b_{t,k}^m.$$

其中, $b_{t,k}^m$ 表示模式 m 在区间 t 启动后, 接下来 k 段内是否出现关闭, 出现时为 1, 否则为 0; $k = 1, \dots, K - 1$ 为检查窗口长度, j 为窗口内的区间偏移。对两种模式、全天各启动位置及检查窗口求和, 得到短段缺额 B 。主方案取 $U = K = 3$, 最短运行约束已使 $B = 0$, 因此该项在主方案中不再提供额外改善, 保留它是为了与允许短动作的经济基准统一比较。

将不同量纲的指标按上界归一化, 得到

$$F_2(x) = \frac{\sum_t \xi_t}{2P_{\max} T} + \frac{N_s}{2T} + \frac{B}{2T(K-1)}. \quad (13)$$

其中， $F_2(x)$ 为方案 x 的归一化运行不平稳程度，无量纲，值越小表示运行越平稳；三个分式依次对应功率总变差、启动次数和短段缺额，分母用于统一数值尺度。三项等权是本文的运行偏好，不表示它们具有相同的寿命损耗。若只允许电费偏离 C^* 一个极小数值容差，第二层可调整的空间有限，故引入相对费用让步 δ ，求解

$$S^* = \min_{x \in \mathcal{X}} F_2(x), \quad F_1(x) \leq (1 + \delta)C^* + \epsilon_C. \quad (14)$$

其中， S^* 为费用预算内第二层目标的最优值，无量纲； $\delta = 0.001$ 为相对第一层最优费用允许的让步比例， $\epsilon_C = 10^{-4}$ 元为数值容差。两者分别控制费用取舍与浮点误差。第二层求解结束后即得到最终方案，不再增加购电量最小化目标。

5.2 问题一模型的求解

通过 Python 依次求解费用和平稳性两个目标，以第二层返回的购电量和充放电轨迹作为最终方案。按题目指定时段汇总购电及储能安排，再复算费用、能量平衡和运行指标，结果分别列于表 2、表 3、表 4 和图 2。

【待更新】以下图表暂保留原计算值。两层结果表中的第二层购电量为 57560.643804 kWh，而后续汇总中的 57560.616077 kWh 来自原第三层，两者尚未统一。全文完成后以两层模型重算，统一替换指定时段、充放电汇总、曲线及比较指标；本次不调整具体数值。

表 2 两层混合整数线性规划的求解结果（待更新）

层次	购电费/元	购电量/kWh	F_2	MIP 间隙
第一层	34080.951400	57572.396296	0.056210610	0
第二层	34115.032451	57560.643804	0.036950810	0

表 3 指定时段购电量及全天购电结果（待更新）

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.000000	12:00–12:10	480.412450	14:00–14:10	0.000000
16:00–16:10	445.431650	18:00–18:10	621.850212	20:00–20:10	2.246095
全天购电量：57560.616077 kWh		全天购电费：34115.032451 元			

表 4 储能分时充放电量与起止储电量（待更新）

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	5059.644256	0.000000	4:00–8:00	0.000000	7357.436146
8:00–12:00	5071.280472	1702.996983	12:00–16:00	4995.867450	0.000000
16:00–20:00	0.000000	6729.235006	20:00–24:00	5059.644256	2378.124655
0:00 储电量: 6000.000000 kWh			24:00 储电量: 6000.000000 kWh		

原结果中，10:00–10:10 和 14:00–14:10 无需购电，说明该方案下光伏与储能已覆盖当段负载；具体时段安排待两层结果更新后核对。四小时内既有充电量又有放电量，是不同十分钟动作相加的结果，并不表示同一时刻同时充放电。

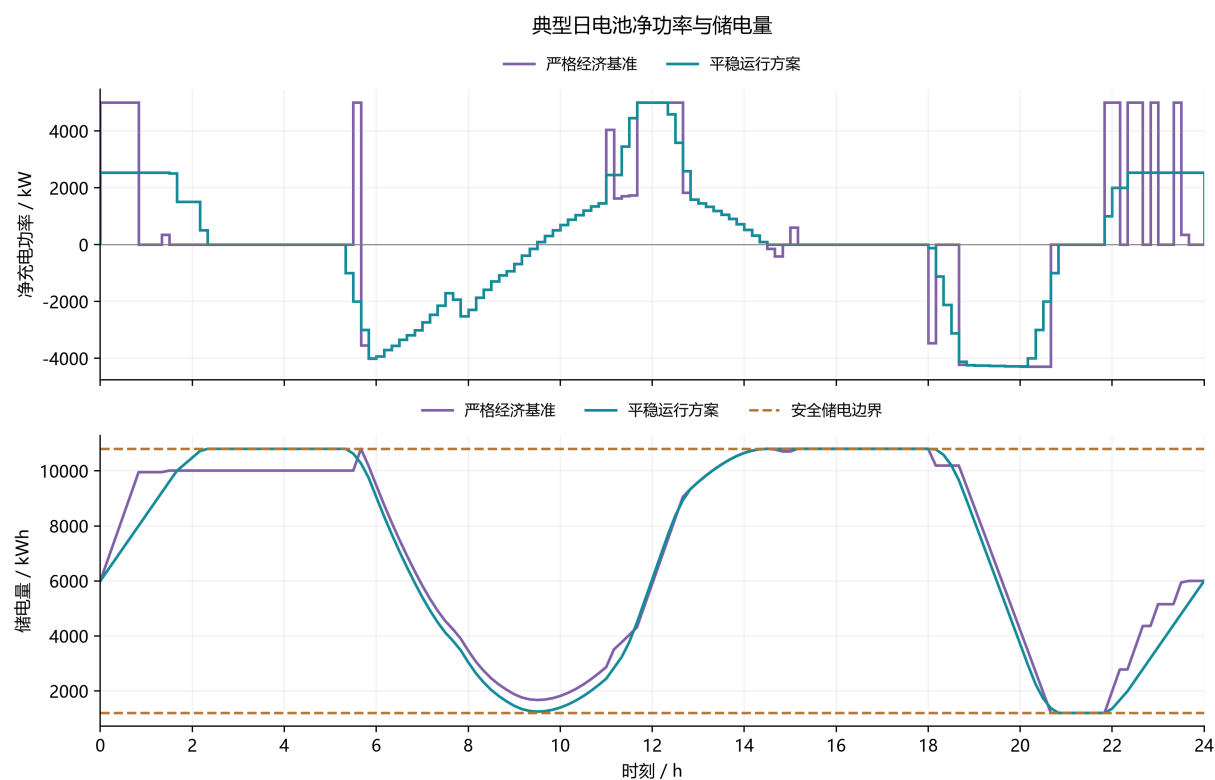


图 2 典型日电池净充电功率与储电轨迹（待更新）

图注：平稳运行方案连接分散的充放电动作，降低相邻功率跳变；储电量始终处于安全区间，并在日末恢复初值。

表 5 储能调度方案的经济性与运行指标对比（待更新）

指标	严格经济基准	平稳运行方案
购电费/元	33801.495642	34115.032451
购电量/kWh	57526.243481	57560.616077
循环功率总变差/kW	102684.580769	33209.310058
最大相邻净功率变化/kW	8546.841200	1000.000000
运行段启动次数/次	12	4
短于 30 分钟的运行段数/段	6	0
转换损耗/kWh	1984.271048	2018.643643

原结果中，平稳方案相较严格经济基准，总变差降低 67.66%，启动次数减少 8 次，购电费增加 313.54 元，增幅约 0.93%。这些数值待两层模型重算后统一核对。比较的重点是减少功率波动需要付出多少费用，而不是直接据此判断电池能够延长多少寿命；短动作减少也同时受到最短运行时间约束的影响。

六、问题二的模型的建立和求解

6.1 具体分析

6.2 模型准备

6.3 模型建立

6.4 模型求解

七、 问题三的模型的建立和求解

7.1 具体分析

7.2 模型准备

7.3 模型建立

7.4 模型求解

八、 问题四的模型的建立和求解

8.1 具体分析

8.2 模型准备

8.3 模型建立

8.4 模型求解

九、模型的分析与检验

9.1 模型检验与敏感性分析

为区分储能收益和运行偏好的代价，设置三个对照：无储能直接购电；不增加爬坡及持续时间要求的严格经济基准；采用 $R_{\text{ramp}} = 1000$ 、 $U = 3$ 、 $D = 2$ 、 $\delta = 0.001$ 的平稳方案。严格经济基准仍进行三层求解，但后层只有数值费用容差。无储能方案为

$$g_t^{(0)} = (L_t - V_t)^+ \Delta t, \quad C^{(0)} = \sum_t p_t g_t^{(0)} = 48052.046591 \text{ 元}. \quad (15)$$

其中， $g_t^{(0)}$ 为无储能条件下区间 t 直接从外网购买的电量，单位为 kWh； $C^{(0)}$ 为该无储能方案的全天购电费用，单位为元；上标 (0) 表示无储能对照方案，取正部表示光伏超过负载时购电量为零。

表 6 储能调度方案的经济性与运行指标对比

指标	严格经济基准	平稳运行方案
购电费/元	33801.495642	34115.032451
购电量/kWh	57526.243481	57560.616077
循环功率总变差/kW	102684.580769	33209.310058
最大相邻净功率变化/kW	8546.841200	1000.000000
运行段启动次数/次	12	4
短于 30 分钟的运行段数/段	6	0
转换损耗/kWh	1984.271048	2018.643643

相较严格经济基准，平稳方案的总变差降低 67.66%，启动次数减少 8 次。购电费增加 313.54 元，增幅约 0.93%，其经济代价可分解为

$$\underbrace{C_{\text{stable}} - C_{\text{econ}}^*}_{\text{总经济代价}} = \underbrace{C^* - C_{\text{econ}}^*}_{\text{新增运行限制}} + \underbrace{C_{\text{stable}} - C^*}_{\text{第二层费用让步}}. \quad (16)$$

其中， C_{stable} 为三层求解后平稳运行方案的实际全天购电费用， C_{econ}^* 为不增加爬坡及最短持续时间限制时严格经济基准的第一层最优费用，单位均为元； C^* 仍为加入运行限制后的第一层最优费用。两项分别约为 279.455857 元和 34.081051 元，其中 C_{econ}^* 为严格经济基准第一层最优值。由此可见，0.1% 指新运行条件下的额外费用预算，而非相对原方案的全部费用增幅。

9.1.1 费用让步比例的选取

固定其他设置，在 $\delta = 0, 0.0005, 0.001, 0.002, 0.005$ 上重新执行三层优化，结果见表 7。将有效运行且净功率绝对值小于 50 kW 的区间称为低功率段，仅用于诊断短小动作，不另作硬约束。

表 7 电费让步比例的单因素重求解结果

$\delta/\%$	购电费/元	购电量/kWh	总变差/kW	启动数	低功率段数
0	34080.951500	57572.396243	50937.422410	6	4
0.05	34097.991975	57570.793390	34345.144627	6	1
0.10	34115.032451	57560.616077	33209.310058	4	0
0.20	34149.113402	57552.981594	31090.138980	4	0
0.50	34251.356257	57516.782839	28513.498282	4	0

由 0 提高至 0.05% 已大幅降低总变差，但启动仍为 6 次；提高至 0.10% 后，启动降至 4 次且无低功率段。继续增加预算仍可降低总变差，却未继续减少启动。因此选取 0.10% 作为本组候选中的折中值，而不将其解释为连续参数空间中的唯一最优点。

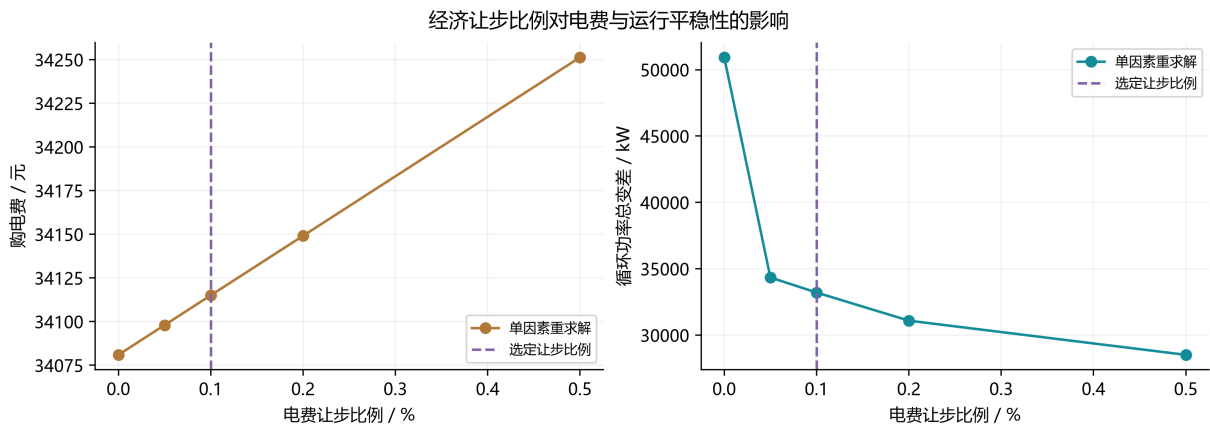


图 3 费用让步比例与运行平稳性的关系

图注：小幅费用让步可明显降低功率总变差；预算继续增加时启动次数不再下降，需结合额外支出选择运行方案。

9.1.2 核心参数的局部扰动检验

对 $\delta, R_{\text{ramp}}, \eta_{\text{rt}}$ 分别作 $\pm 10\%$ 相对扰动，其余参数不变，每种情景均重新求解三个层次。定义指标 J 的相对变化及局部灵敏度为

$$D_J(\theta') = \frac{J(\theta') - J(\theta)}{J(\theta)} \times 100\%, \quad S_J(\theta') = \frac{[J(\theta') - J(\theta)]/J(\theta)}{(\theta' - \theta)/\theta}. \quad (17)$$

其中， θ 为被扰动参数的基准值， θ' 为扰动后的取值； J 为所考察的输出指标，如购电费或功率总变差， $J(\theta)$ 与 $J(\theta')$ 分别为扰动前、后重新求解得到的指标值。 $D_J(\theta')$ 为该

指标的百分比变化, $S_J(\theta')$ 为指标相对变化与参数相对变化之比, 是无量纲的有限扰动灵敏度。整数模式可能跳变, 故 S_J 用于有限扰动比较, 不等同于处处存在的导数。参数表中 δ 的 $\pm 10\%$ 是 0.09% 与 0.11%, 效率的对应值是 81% 与 99%, 后者为机制压力情景, 不表示实测效率误差范围。

表 8 核心参数相对变化 10% 时的局部敏感性

参数情景	购电费/元	费用变化/%	总变差/kW	启动数
$\delta = 0.09\%$	34111.6244	-0.0100	34436.1963	4
$\delta = 0.11\%$	34118.4405	0.0100	32900.5165	4
$R_{\text{ramp}} = 900 \text{ kW}$	34153.5161	0.1128	33045.9374	4
$R_{\text{ramp}} = 1100 \text{ kW}$	34085.2443	-0.0873	33952.8546	4
$\eta_{\text{rt}} = 81\%$	35470.6053	3.9735	32667.2116	4
$\eta_{\text{rt}} = 99\%$	32897.4785	-3.5690	33591.6607	6

费用让步比例的局部变化仅使电费变动约 $\pm 0.0100\%$, 启动数均为 4; 但总变差与低功率段仍会变化, 说明局部轨迹并非完全不敏感。爬坡限值由 1000 降低至 900 kW, 费用增加约 0.1128%; 放宽至 1100 kW, 费用下降约 0.0873%, 体现了连续运行与经济支出的权衡。

相比之下, 往返效率降至 81% 时费用增加约 3.9735%, 升至 99% 时费用下降约 3.5690%, 且后者启动次数增至 6。效率影响单位放电的购入成本, 还会改变有利套利时段和模式选择。因此只能认为本轮方案对费用预算及爬坡参数的局部扰动较稳定, 不能据此声称对全部参数均不敏感。各情景的独立约束违反项均为 0。

9.1.3 输入噪声与固定计划压力测试

在缺乏实测误差分布的条件下, 设置小幅、可重复的数值扰动, 检验模型对输入变化的反应:

$$\begin{aligned}\tilde{L}_t &= L_t(1 + \zeta_t^L), & \tilde{V}_t &= V_t(1 + \zeta_t^V), \\ \zeta_t^j &= \text{clip}(Z_t^j, -0.03, 0.03), & Z_t^j &\sim N(0, 0.01^2).\end{aligned}\tag{18}$$

其中, $\tilde{L}_t$ 与 $\tilde{V}_t$ 分别为扰动后的负载功率与光伏功率, 单位为 kW; ζ_t^L 与 ζ_t^V 为相应的无量纲相对扰动。此式 $j \in \{L, V\}$ 表示扰动对象, Z_t^j 为均值 0、标准差 0.01 的正态随机扰动, $N(0, 0.01^2)$ 中的第二个参数为方差; clip 将扰动截断在 $[-0.03, 0.03]$, 即 $\pm 3\%$ 以内。使用种子 2026、2027、2028, 保持电价不变、夜间零光伏不变, 各情景独立重求解。同时冻结原购电和充放电指令, 计算其不能覆盖的正向净负荷扰动:

$$H_{\text{fixed}} = \sum_t \left[(\tilde{L}_t - \tilde{V}_t - L_t + V_t) \Delta t - w_t \right]^+.\tag{19}$$

其中， H_{fixed} 为冻结原购电及充放电指令后，全天各区间供电缺口的累计电量，单位为 kWh；方括号内为净负荷新增电量减去原方案剩余电量 w_t ，外侧取正部表示只累计未能覆盖的缺口，不用其他区间的富余电量直接抵消。 H_{fixed} 是不允许重新调度或补购时的计划缺口，不是采用实时补救后必然发生的停电量。两项检验分别回答“重新优化是否稳定”与“固定计划能否直接应对扰动”。

表 9 1% 尺度输入扰动下的重求解与固定计划检验

随机种子	重求解费用/元	费用变化/%	固定计划缺口/kWh	重求解违反项
2026	34143.6909	0.0840	609.1943	0
2027	34304.0402	0.5540	723.4288	0
2028	34058.5619	-0.1655	470.7939	0

三次重求解的费用变化处于 -0.1655% 至 0.5540% ，全部通过约束复核，说明本组小幅扰动下重新规划仍能得到接近原费用的可行解。然而，冻结原计划后累计出现 $470.7939\text{--}723.4288\text{ kWh}$ 的区间缺口，表明确定性计划不具备自动抵御供需误差的保供保证。该结果也说明，后续引入实时补购与滚动调整具有必要性。三次独立高斯扰动仅检验局部数值稳定性，不代表真实误差具有独立正态分布，也不提供极端天气或相关预测偏差下的概率保证。

9.2 灵敏度分析

9.3 误差分析

十、模型的评价、改进与推广

10.1 模型的优点

- 优点 1
- 优点 2
- 优点 3

10.2 模型的缺点

- 缺点 1
- 缺点 2

10.3 模型的改进

10.4 模型的推广

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型思路讨论、程序编写与调试、结果分析辅助、论文表述与排版调整，详细使用情况见支撑材料。

参考文献

- [1] BABIĆ L, LAURICELLA M, CEUSTERS G, et al. Data-driven non-parametric chance-constrained model predictive control for microgrids energy management using small data batches[J]. *Frontiers in Control Engineering*, 2023, 4: 1237759. DOI: <https://doi.org/10.3389/fcteg.2023.1237759>.
- [2] GESLIN A, XU L, GANAPATHI D, et al. Dynamic cycling enhances battery lifetime[J]. *Nature Energy*, 2025, 10: 172–180. DOI: <https://doi.org/10.1038/s41560-024-01675-8>.
- [3] SciPy contributors. scipy.optimize.milp: Mixed-integer linear programming[EB/OL]. SciPy 1.15.3 documentation. <https://docs.scipy.org/doc/scipy-1.15.3/reference/generated/scipy.optimize.milp.html>.

附录 A 文件列表

文件名	功能描述
q1_reproduce.py	问题一完整求解、检验与绘图程序（含内嵌数据）
数据预处理.py	预处理程序代码
q2.py	问题二程序代码
q3.c	问题三程序代码
q4.cpp	问题四程序代码
AI 工具使用详情.pdf	AI 工具、使用过程及人工修改与核验情况说明（提交前补齐）

附录 B 问题一完整求解与检验程序

本程序同时包含原始输入、三层求解、独立约束检验、参数敏感性、噪声压力测试及绘图。数据取自附件一的 144 段记录，程序不依赖个人绝对路径。将以下代码保存为 `q1_reproduce.py`，安装依赖后直接执行：

```
python -m pip install numpy scipy matplotlib
python q1_reproduce.py
```

若希望读取外部 Excel，另安装 `openpyxl`，并使用 `--input` 传入文件名。外部文件损坏或缺失时默认停止，只有显式加入 `--fallback` 才回退到内嵌数据，避免悄悄替换输入。

`--quick` 只运行主方案与严格经济基准；默认运行 12 组参数方案和 3 组噪声方案。

输出目录默认为 `q1_output`，其中 `revised.csv` 包含 144 段完整购电、充放电及储电量；各情景的 JSON 文件保存三层求解状态与完整轨迹，`summary.json` 包含指定时段、四小时汇总及检验结果。CSV 可直接用表格软件打开，输入附件和已有结果工作簿均不修改。两幅 PNG 图由同一批计算结果绘制，正文采用本次完整运行的输出。

```
1 """问题一完整复现: Python >=3.10; pip install numpy scipy matplotlib.
2 直接运行 python q1_reproduce.py; 输入内嵌为附件1原始144行。
3 可选 --input 附件1.csv / 附件1.xlsx; 损坏文件默认报错, --fallback才使用内嵌数据。
4 全部输出放入 --out 指定的新目录; 不覆盖原始附件或队友工作簿。
5 """
6 import argparse # 解析命令行参数。
7 import csv # 读写不依赖Excel软件的表格。
8 import io # 将内嵌文本作为文件读取。
9 import json # 保存完整数值和求解状态。
10 import os # 将绘图缓存限制在本次输出目录。
11 import sys # 向终端返回失败状态。
12 import time # 记录真实求解耗时。
13 from pathlib import Path # 使用跨平台路径, 不写入个人绝对路径。
14 try:
15     import numpy as np # 数值计算。
16     import scipy # 记录实际使用的库版本。
17     from scipy.optimize import milp, Bounds, LinearConstraint # 混合整数规划。
18     from scipy.sparse import coo_matrix # 稀疏约束矩阵。
19 except ImportError as exc:
20     raise SystemExit("请先安装: pip install numpy scipy matplotlib") from exc
21
22 # 以下是附件1逐行数据, 顺序为区间右端、电价、负载、光伏功率。
23 EMBEDDED_CSV = """时间,电价,小区负载,光伏发电预测功率
24 00:10:00,0.4248,3439.8466,0
25 00:20:00,0.4245,3437.9792,0
26 00:30:00,0.4269,3444.3031,0
27 00:40:00,0.4272,3454.4437,0
28 00:50:00,0.4271,3458.8273,0
29 01:00:00,0.4285,3459.8401,0
30 01:10:00,0.4311,3467.1582,0
31 01:20:00,0.4302,3467.9632,0
32 01:30:00,0.4279,3466.9683,0
33 01:40:00,0.4312,3482.4927,0
34 01:50:00,0.4333,3495.0841,0
35 02:00:00,0.4311,3498.4443,0
36 02:10:00,0.4293,3505.2295,0
37 02:20:00,0.4324,3512.6387,0
38 02:30:00,0.4367,3516.8685,0
39 02:40:00,0.4348,3517.9246,0
40 02:50:00,0.4335,3518.5518,0
41 03:00:00,0.4357,3521.0976,0
42 03:10:00,0.4374,3526.9261,0
43 03:20:00,0.4371,3537.2854,0
44 03:30:00,0.4356,3544.8808,0
```

45	03:40:00,0.4353,3541.2787,0
46	03:50:00,0.436,3546.3976,0
47	04:00:00,0.4378,3560.207,0
48	04:10:00,0.4401,3561.9581,0
49	04:20:00,0.4389,3556.7183,0
50	04:30:00,0.4364,3560.0272,0
51	04:40:00,0.4393,3580.654,0.5994
52	04:50:00,0.4422,3586.916,0.2345
53	05:00:00,0.4395,3571.2635,0.2885
54	05:10:00,0.4333,3562.3171,18.6781
55	05:20:00,0.4439,3591.3809,54.4698
56	05:30:00,0.4301,3579.7045,108.9718
57	05:40:00,0.3713,3486.119,184.1603
58	05:50:00,0.5708,3823.8994,277.0582
59	06:00:00,0.9099,4398.0727,388.5494
60	06:10:00,0.9368,4454.0123,521.2803
61	06:20:00,0.8758,4371.7919,672.7677
62	06:30:00,0.8759,4392.953,844.2261
63	06:40:00,0.8601,4384.5956,1039.6744
64	06:50:00,0.8442,4434.7757,1259.3375
65	07:00:00,0.8334,4523.4561,1508.6318
66	07:10:00,0.8092,4524.0721,1791.4107
67	07:20:00,0.7972,4545.8867,2085.4865
68	07:30:00,0.7644,4523.2418,2381.2611
69	07:40:00,0.6899,4383.3566,2683.1752
70	07:50:00,0.86,4910.4599,2986.3552
71	08:00:00,1.1617,5799.7327,3289.204
72	08:10:00,1.1683,5884.826,3593.3613
73	08:20:00,1.0956,5748.0713,3890.7332
74	08:30:00,1.0849,5769.0581,4181.3417
75	08:40:00,1.061,5754.2105,4471.7657
76	08:50:00,1.043,5824.1807,4752.8272
77	09:00:00,1.036,5954.2601,5021.9074
78	09:10:00,1.0147,5958.9696,5285.4126
79	09:20:00,0.9922,5929.5554,5543.1742
80	09:30:00,0.97,5923.7828,5788.4093
81	09:40:00,0.9422,5907.586,6012.1304
82	09:50:00,0.9569,5899.1262,6216.1528
83	10:00:00,0.9977,5902.2196,6409.5941
84	10:10,0.9919,5897.4793,6599.3808
85	10:20,0.9588,5886.7568,6766.7204
86	10:30,0.9549,5873.4708,6914.5923
87	10:40,0.9839,5858.0308,7064.2053
88	10:50,0.799,5844.9948,7197.4315
89	11:00,0.5025,5839.0587,7303.1142
90	11:10,0.4662,5838.7103,7387.6848
91	11:20,0.5007,5823.4378,7459.227
92	11:30,0.4827,5814.7932,7520.7191
93	11:40,0.4785,5832.3951,7568.2039
94	11:50,0.4662,5712.901,7592.1539
95	12:00,0.4432,5519.4607,7601.0433
96	12:10,0.4411,5494.7907,7612.316
97	12:20,0.4551,5514.4524,7602.6524
98	12:30,0.4463,5485.829,7554.1872
99	12:40,0.4113,5462.3628,7472.7384
100	12:50,0.5682,5566.1066,7393.4172
101	13:00,0.8279,5725.7548,7312.7979
102	13:10,0.8636,5739.4957,7194.541
103	13:20,0.8416,5714.6836,7053.1371
104	13:30,0.8618,5717.8505,6910.4064
105	13:40,0.8623,5708.2486,6772.4075
106	13:50,0.9026,5696.9182,6607.36
107	14:00,0.9754,5692.2983,6412.5085
108	14:10,1.0002,5687.1291,6214.0158
109	14:20,0.9978,5678.962,6000.7112
110	14:30,1.0271,5669.4508,5766.9816
111	14:40,1.0906,5664.1948,5521.5066
112	14:50,0.9376,5672.7011,5268.781
113	15:00,0.6694,5682.2544,5009.7152
114	15:10,0.66,5669.0195,4743.4484

```

115 15:20,0.7217,5656.3651,4471.224
116 15:30,0.7308,5657.0867,4188.2025
117 15:40,0.7534,5662.6008,3889.3931
118 15:50,0.7823,5669.2596,3594.5946
119 16:00,0.8055,5673.4767,3303.8878
120 16:10,0.8271,5671.0146,2998.4247
121 16:20,0.8519,5661.5926,2687.4131
122 16:30,0.8791,5665.8922,2383.1779
123 16:40,0.9024,5689.0372,2089.1246
124 16:50,0.9163,5603.6614,1791.0829
125 17:00,0.9264,5466.7124,1501.7283
126 17:10,0.9462,5467.3939,1255.1723
127 17:20,0.9742,5464.727,1036.3323
128 17:30,0.9862,5492.4458,836.7775
129 17:40,0.976,5621.7308,665.2313
130 17:50,1.0742,5091.2126,516.6758
131 18:00,1.2313,4208.5688,386.6731
132 18:10,1.2561,4122.7611,275.7933
133 18:20,1.2447,4255.9358,182.9526
134 18:30,1.2545,4225.9524,108.3414
135 18:40,1.2522,4233.3384,54.3121
136 18:50,1.2881,4246.4625,18.6569
137 19:00,1.3466,4239.1795,0.2753
138 19:10,1.3523,4244.6684,0.2296
139 19:20,1.3445,4251.9187,0.6007
140 19:30,1.3483,4256.6813,0
141 19:40,1.349,4267.3947,0
142 19:50,1.3473,4269.975,0
143 20:00,1.3482,4268.6702,0
144 20:10,1.3513,4282.2245,0
145 20:20,1.34,4291.7049,0
146 20:30,1.3489,4291.3611,0
147 20:40,1.3952,4293.9182,0
148 20:50,1.2226,4295.2135,0
149 21:00,0.9332,4302.0307,0
150 21:10,0.9068,4320.9618,0
151 21:20,0.9371,4302.3292,0
152 21:30,0.9371,4322.6758,0
153 21:40,0.9871,4428.8083,0
154 21:50,0.7764,4030.1614,0
155 22:00,0.4209,3362.7419,0
156 22:10,0.3859,3309.3934,0
157 22:20,0.4368,3412.9563,0
158 22:30,0.4198,3381.9313,0
159 22:40,0.4186,3386.5592,0
160 22:50,0.4241,3400.4846,0
161 23:00,0.4237,3399.2426,0
162 23:10,0.4248,3406.9809,0
163 23:20,0.424,3416.1518,0
164 23:30,0.422,3421.9811,0
165 23:40,0.424,3429.6923,0
166 23:50,0.4271,3439.061,0
167 0:00+1,0.4276,3444.7259,0""
168 T, DT = 144, 1 / 6 # 一天144个10分钟区间, 功率转电量乘DT。
169 PMAX, PMIN, EMIN, EMAX, EINIT = 5000., 1., 1200., 10800., 6000.
170 EPS_C, EPS_S = 1e-4, 1e-7 # 费用和无量纲第二目标的绝对容差。
171
172
173 def read_data(path=None, fallback=False):
174     """只读输入; 任何缺失、负数、时间错位都显式报错。"""
175     try:
176         if path is None:
177             rows = list(csv.reader(io.StringIO(EMBEDDED_CSV.strip())))[1:]
178         elif Path(path).suffix.lower() == ".xlsx":
179             from openpyxl import load_workbook # 仅外部XLSX输入需要该可选库。
180             book = load_workbook(path, data_only=True, read_only=True)
181             rows = list(book.active.values)[1:] # 跳过标题; 不会保存或修改源文件。
182             book.close()
183         else:
184             rows = None

```



```

185         for encoding in ("utf-8-sig", "gb18030"): # 兼容常用中文CSV编码。
186             try:
187                 with open(path, encoding=encoding, newline="") as handle:
188                     rows = list(csv.reader(handle))[1:]
189                     break
190             except UnicodeDecodeError:
191                 continue
192         if rows is None:
193             raise ValueError("CSV编码无法识别")
194         if len(rows) != T:
195             raise ValueError(f"应有144行数据, 实际{len(rows)}行")
196         for i, row in enumerate(rows): # 检查右端时刻, 避免错位十分钟。
197             label = str(row[0]).strip()
198             if i == T - 1 and label in ("0:00+1", "00:00+1", "24:00", "24:00:00", "00:00:00"):
199                 continue
200             pieces = label.split(":")
201             if len(pieces) < 2 or int(pieces[0]) * 60 + int(pieces[1]) != (i + 1) * 10:
202                 raise ValueError(f"第{i+1}行时间不连续: {label}")
203         data = np.asarray([[float(x) for x in row[1:4]] for row in rows])
204         if data.shape != (T, 3) or not np.isfinite(data).all():
205             raise ValueError("存在空值、非数值或非有限数")
206         if (data < 0).any() or (data[:, 0] <= 0).any():
207             raise ValueError("本模型要求正电价以及非负负载、光伏")
208         return data
209     except Exception as exc: # 同时兼容Excel压缩包损坏、编码和格式解析异常。
210         if path is not None and fallback:
211             print(f"警告: 读取失败({exc}), 明确回退到内嵌附件1", file=sys.stderr)
212             return read_data()
213         raise ValueError(f"输入读取失败: {exc}; 可用--fallback允许内嵌回退") from exc
214
215
216 def solve(data, delta=.001, ramp=1000., up=3, down=2, eta_rt=.9):
217     """同一可行域内依次求费用、平稳性、购电量; 全部层均保留整数互斥。"""
218     if not (0 < eta_rt <= 1 and delta >= 0 and 1 <= up <= T and 1 <= down <= T):
219         raise ValueError("效率、让步比例或持续时间不合法")
220     eta = np.sqrt(eta_rt) # 本问保持往返效率口径。
221     names = ("g", "c", "d", "w", "E", "zc", "zd", "sc", "sd", "oc", "od", "v",
222             "bc1", "bc2", "bd1", "bd2") # b为旧基准短段缺额线性化变量。
223     ids, offset = {}, 0
224     for name in names:
225         size = T + 1 if name == "E" else T # 储电量包含145个边界。
226         ids[name] = np.arange(offset, offset + size)
227         offset += size
228     lo, hi, integer = np.zeros(offset), np.full(offset, np.inf), np.zeros(offset)
229     for name in ("c", "d"):
230         hi[ids[name]] = PMAX
231     lo[ids["E"]], hi[ids["E"]] = EMIN, EMAX
232     lo[ids["E"]][0, -1]] = EINIT
233     hi[ids["E"]][0, -1]] = EINIT # 日初日末均6000kWh。
234     for name in ("zc", "zd", "sc", "sd", "oc", "od", "bc1", "bc2", "bd1", "bd2"):
235         hi[ids[name]] = 1
236     integer[ids["zc"]], integer[ids["zd"]] = 1, 1 # 仅模式变量必须声明整数。
237     ri, ci, av, lower, upper = [], [], [], [], []
238     def add(items, lb=-np.inf, ub=np.inf):
239         """将一行稀疏线性约束加入模型。"""
240         row = len(lower)
241         for col, val in items:
242             ri.append(int(col)); av.append(float(val))
243         lower.append(float(lb)); upper.append(float(ub))
244     for t in range(T):
245         prev = (t - 1) % T # 循环日跨午夜连接。
246         ix = lambda key, j=t: int(ids[key][j])
247         net = [(ix("c"), 1), (ix("d"), -1), (ix("c", prev), -1), (ix("d", prev), 1)]
248         rhs = (data[t, 1] - data[t, 2]) * DT
249         add([(ix("g"), 1), (ix("d"), DT), (ix("c"), -DT), (ix("w"), -1)], rhs, rhs)
250         add([(ix("E", t+1), 1), (ix("E"), -1), (ix("c"), -eta*DT), (ix("d"), DT/eta)], 0, 0)
251         add([(ix("zc"), 1), (ix("zd"), 1)], ub=1) # 禁止同时充放电。
252         add(net, -ramp, ramp) # 运行功率变化上限, 基准可设为10000。
253         add([(ix("v"), 1)] + [(k, -v) for k, v in net], lb=0)
254         add([(ix("v"), 1)] + net, lb=0) # v不小于净功率差的绝对值。

```

```

255     for mode in ("c", "d"):
256         z, oldz, start, stop = ix("z"+mode), ix("z"+mode, prev), ix("s"+mode), ix("o"+mode)
257         add([(ix(mode), 1), (z, -PMAX)], ub=0)
258         add([(ix(mode), 1), (z, -PMIN)], lb=0)
259         add([(start, 1), (z, -1), (oldz, 1)], lb=0)
260         add([(start, 1), (z, -1)], ub=0)
261         add([(start, 1), (oldz, 1)], ub=1)
262         add([(stop, 1), (oldz, -1), (z, 1)], lb=0)
263         add([(stop, 1), (oldz, -1)], ub=0)
264         add([(stop, 1), (z, 1)], ub=1) # s、o精确表示0-1转换。
265         add([(ix("z"+mode, (t+j)%T), 1) for j in range(up)] + [(start, -up)], lb=0)
266         add([(ix("z"+mode, (t+j)%T), 1) for j in range(down)] + [(stop, down)], ub=down)
267     for k in (1, 2): # 30分钟参考长度的两级短段缺额。
268         b = ix("b"+mode+str(k))
269         add([(b, 1), (start, -1)], ub=0)
270         for j in range(1, k+1):
271             add([(b, 1), (start, -1), (ix("z"+mode, (t+j)%T), 1)], lb=0)
272             add([(b, 1)] + [(ix("z"+mode, (t+j)%T), 1) for j in range(1, k+1)], ub=k)
273 cost, smooth, quantity = np.zeros(offset), np.zeros(offset), np.zeros(offset)
274 cost[ids["g"]], quantity[ids["g"]] = data[:, 0], 1
275 smooth[ids["v"]] = 1 / (2 * PMAX * T)
276 for name in ("sc", "sd"):
277     smooth[ids[name]] = 1 / (2 * T)
278 for name in ("bc1", "bc2", "bd1", "bd2"):
279     smooth[ids[name]] = 1 / (4 * T)
280 records, solutions = [], []
281 for stage, objective in enumerate((cost, smooth * 1e6, quantity), 1):
282     matrix = coo_matrix((av, (ri, ci)), shape=(len(lower), offset)).tocsc()
283     began = time.perf_counter()
284     result = milp(objective, integrality=integer, bounds=Bounds(lo, hi),
285                  constraints=LinearConstraint(matrix, lower, upper),
286                  options={"mip_rel_gap": 1e-9, "time_limit": 180.})
287     if not result.success or result.x is None:
288         raise RuntimeError(f"第{stage}层未证最优: {result.message}")
289     x = result.x # 不对连续解取整, 不静默接受超时解。
290     rec = dict(stage=stage, cost=float(cost@x), quantity=float(quantity@x),
291              smooth=float(smooth@x), gap=float(result.mip_gap),
292              seconds=time.perf_counter()-began, nodes=int(result.mip_node_count))
293     records.append(rec); solutions.append({k: x[v] for k, v in ids.items()})
294     if stage == 1:
295         budget = (1 + delta) * rec["cost"] + EPS_C
296         add(list(zip(ids["g"], data[:, 0])), ub=budget)
297     elif stage == 2:
298         add([(i, v*1e6) for i, v in enumerate(smooth) if v], ub=(rec["smooth"]+EPS_S)*1e6)
299     config = dict(delta=delta, ramp=ramp, up=up, down=down, eta_rt=eta_rt)
300     result = dict(config=config, stages=records, trajectory=solutions[-1], budget=budget)
301     result["metrics"] = audit(data, result) # 从物理量独立复算, 不只看求解器成功标志。
302     return result
303
304
305 def lengths(z, value=1):
306     """读取循环0-1序列的段长, 合并跨午夜的同模式段。"""
307     starts = [i for i in range(T) if z[i] == value and z[(i-1)%T] != value]
308     if not starts:
309         return [T] if np.all(z == value) else []
310     return [next(k for k in range(1, T+1) if z[(i+k)%T] != value) for i in starts]
311
312
313 def audit(data, result):
314     """独立核验供需、SOC、功率、互斥、段长、费用预算及层次目标。"""
315     q, cfg = result["trajectory"], result["config"]
316     g, c, d, w, E = (q[k] for k in ("g", "c", "d", "w", "E"))
317     eta = np.sqrt(cfg["eta_rt"])
318     net = c-d
319     on, off = [], []
320     for key in ("zc", "zd"):
321         z = np rint(q[key]).astype(int) # 仅用于诊断整数变量, 不更改求解值。
322         on.extend(lengths(z)); off.extend(lengths(z, 0))
323     balance = g + data[:, 2]*DT + d*DT - data[:, 1]*DT - c*DT - w
324     transition = np.diff(E)-eta*c*DT+d*DT/eta

```

```

325 tv = float(np.abs(net-np.roll(net, 1)).sum())
326 short = sum(max(0, 3-k) for k in on)
327 smooth = tv/(2*PMAX*T)+len(on)/(2*T)+short/(4*T)
328 metrics = dict(cost=float(data[:, 0][g]), quantity=float(g.sum()), tv=tv, starts=len(on),
329               short_deficit=short, short_runs=sum(k<3 for k in on),
330               min_on=min(on, default=T)*10, min_off=min(off, default=T)*10,
331               max_step=float(np.abs(net-np.roll(net, 1)).max()),
332               charge=float(c.sum()*DT), discharge=float(d.sum()*DT),
333               surplus=float(w.sum()), emin=float(E.min()), emax=float(E.max()),
334               balance=float(np.abs(balance).max()), state=float(np.abs(transition).max()),
335               endpoint=float(max(abs(E[0]-EINIT), abs(E[-1]-EINIT))),
336               simultaneous=float(np.minimum(c, d).max()), smooth=smooth,
337               max_c=float(c.max()), max_d=float(d.max()),
338               low=int(np.sum((np.abs(net)>1e-5)&(np.abs(net)<50))))
339 metrics["daily_balance"] = float(abs(g.sum() - (data[:, 1]-data[:, 2]).sum()*DT
340                                - (c.sum()-d.sum()*DT - w.sum()))
341 metrics["cost_recompute"] = abs(metrics["cost"]-result["stages"][-1]["cost"])
342 tolerance = 2e-5 # kW/kWh诊断容差, 目标函数另用各自量尺核查。
343 violations = [
344     metrics["balance"]>tolerance, metrics["state"]>tolerance,
345     metrics["endpoint"]>tolerance, metrics["simultaneous"]>tolerance,
346     E.min()<EMIN-tolerance, E.max()>EMAX+tolerance,
347     c.max()>PMAX+tolerance, d.max()>PMAX+tolerance,
348     min(g.min(), c.min(), d.min(), w.min()) < -tolerance,
349     metrics["max_step"]>cfg["ramp"]+tolerance,
350     metrics["min_on"]<cfg["up"]*10, metrics["min_off"]<cfg["down"]*10,
351     metrics["cost"]>result["budget"]+1e-5,
352     smooth>result["stages"][1]["smooth"]+EPS_S+1e-9,
353     np.max(np.abs(q["zc"]-np rint(q["zc"])))>1e-6,
354     np.max(np.abs(q["zd"]-np rint(q["zd"])))>1e-6,
355     np.max(q["zc"]+q["zd"])>1+1e-6,
356     np.max(PMIN*q["zc"]-c)>tolerance, np.max(c-PMAX*q["zc"])>tolerance,
357     np.max(PMIN*q["zd"]-d)>tolerance, np.max(d-PMAX*q["zd"])>tolerance,
358     metrics["daily_balance"]>tolerance, metrics["cost_recompute"]>1e-5,
359 ]
360 metrics["violations"] = sum(bool(v) for v in violations)
361 if metrics["violations"]:
362     raise RuntimeError(f"独立约束检验失败: {metrics}")
363 return metrics
364
365
366 def save_case(out, name, data, result):
367     """导出144段完整计划及每层状态, 不覆盖用户原始表。"""
368     def serial(value):
369         if isinstance(value, np.ndarray):
370             return value.tolist()
371         if isinstance(value, np.generic):
372             return value.item()
373         raise TypeError(f"无法序列化: {type(value)}")
374     (out/(name+".json")).write_text(json.dumps(result, ensure_ascii=False, indent=2,
375                                               default=serial), encoding="utf-8")
376     q = result["trajectory"]
377     with (out/(name+".csv")).open("w", newline="", encoding="utf-8-sig") as handle:
378         writer = csv.writer(handle)
379         writer.writerow(["起点/h", "终点/h", "电价/元每kWh", "负载/kW", "光伏/kW",
380                         "购电量/kWh", "充电功率/kW", "放电功率/kW", "剩余/kWh",
381                         "期初储电/kWh", "期末储电/kWh", "区间电费/元"])
382         for t in range(T):
383             writer.writerow([t*DT, (t+1)*DT, *data[t], q["g"][t], q["c"][t], q["d"][t],
384                             q["w"][t], q["E"][t], q["E"][t+1], data[t, 0]*q["g"][t]])
385
386
387 def draw(out, data, cases, sweep):
388     """依据真实解绘制阶梯功率、储电轨迹和费用-总变差图。"""
389     os.environ.setdefault("MPLCONFIGDIR", str((out/"mpl-cache").resolve()))
390     import matplotlib
391     matplotlib.use("Agg") # 无图形界面的机器也能输出。
392     import matplotlib.pyplot as plt
393     plt.rcParams.update({"font.sans-serif": ["Microsoft YaHei", "SimHei", "DejaVu Sans"],
394                         "axes.unicode_minus": False, "font.size": 10,

```

```

395         "axes.spines.top": False, "axes.spines.right": False})
396 edges = np.arange(T+1)*DT
397 fig, axes = plt.subplots(2, 1, figsize=(10, 6.4), sharex=True, layout="constrained")
398 for key, label, color in [("baseline", "严格经济基准", "#8061A8"), ("revised", "平稳运行方案", "#138B99")]:
399     q = cases[key]["trajectory"]
400     axes[0].stairs(q["c"]-q["d"], edges, label=label, color=color, linewidth=1.6)
401     axes[1].plot(edges, q["E"], label=label, color=color, linewidth=1.6)
402 axes[0].axhline(0, color="#777777", linewidth=.6)
403 axes[0].set_ylabel("净充电功率 / kW") # 标题与图例分别留出空间。
404 axes[0].set_title("典型日电池净功率与储电量", pad=34)
405 axes[1].axhline(EMIN, color="#B27838", linestyle="--", label="安全储电边界")
406 axes[1].axhline(EMAX, color="#B27838", linestyle="--")
407 axes[1].set_xlabel="时刻 / h", ylabel="储电量 / kWh", xlim=(0,24), xticks=np.arange(0,25,2))
408 for ax in axes:
409     ax.legend(ncol=3, fontsize=9, loc="lower center",
410             bbox_to_anchor=(.5, 1.01), frameon=False) # 图例不遮挡数据轨迹。
411     ax.grid(alpha=.16)
412 fig.savefig(out/"q1_dispatch.png", dpi=320); plt.close(fig)
413 rows = sorted(sweep, key=lambda row: row["delta"])
414 fig, axes = plt.subplots(1, 2, figsize=(10, 3.5), layout="constrained")
415 x = [row["delta"]*100 for row in rows]
416 for ax, key, ylabel, color in [(axes[0], "cost", "购电费 / 元", "#B27838"),
417                               (axes[1], "tv", "循环功率总变差 / kW", "#138B99")]:
418     ax.plot(x, [row[key] for row in rows], "o-", color=color, label="单因素重求解")
419     ax.axvline(.1, color="#8061A8", linestyle="--", label="选定让步比例")
420     ax.set_xlabel="电费让步比例 / %", ylabel=ylabel; ax.grid(alpha=.16); ax.legend(fontsize=8)
421 fig.suptitle("经济让步比例对电费与运行平稳性的影响")
422 fig.savefig(out/"q1_tradeoff.png", dpi=320); plt.close(fig)
423
424
425 def main():
426     parser = argparse.ArgumentParser(description=__doc__)
427     parser.add_argument("--input", help="可选CSV或XLSX; 省略则读取内嵌附件1")
428     parser.add_argument("--fallback", action="store_true", help="外部输入失败时显式回退")
429     parser.add_argument("--out", default="q1_output", help="生成结果目录")
430     parser.add_argument("--quick", action="store_true", help="只做主方案和严格经济基准")
431     args = parser.parse_args()
432     data = read_data(args.input, args.fallback)
433     out = Path(args.out); out.mkdir(parents=True, exist_ok=True)
434     cases, sweep, sensitivity = {}, [], []
435     tasks = [("revised", {}), ("baseline", dict(delta=0, ramp=10000, up=1, down=1))]
436     if not args.quick:
437         tasks += [(f"delta_{v}", dict(delta=v)) for v in (0, .0005, .002, .005)]
438         tasks += [(f"{key}_{factor}", {key: value*factor}) for key, value in
439                 [("delta", .001), ("ramp", 1000.), ("eta_rt", .9)] for factor in (.9, 1.1)]
440     for name, settings in tasks:
441         print(f"求解 {name}", flush=True)
442         result = solve(data, **settings)
443         cases[name] = result; save_case(out, name, data, result)
444         row = dict(name=name, **result["config"], **result["metrics"])
445         sensitivity.append(row)
446         if name == "revised" or name.startswith("delta_") and name not in ("delta_0.9", "delta_1.1"):
447             :
448             sweep.append(row)
449             print(json.dumps(row, ensure_ascii=False), flush=True)
450     noise = []
451     if not args.quick:
452         for seed in range(2026, 2029): # 三次局部压力测试, 不宣称统计置信保证。
453             rng = np.random.default_rng(seed)
454             perturbed = data.copy()
455             factors = 1 + np.clip(rng.normal(0, .01, (T,2)), -.03, .03)
456             perturbed[:, 1:] *= factors # 夜间零光伏保留为零。
457             result = solve(perturbed)
458             save_case(out, f"noise_{seed}", perturbed, result)
459             fixed = cases["revised"]["trajectory"]
460             gap = ((perturbed[:, 1]-perturbed[:, 2])-(data[:, 1]-data[:, 2]))*DT-fixed["w"]
461             row = dict(seed=seed, cost=result["metrics"]["cost"],
462                     change_pct=(result["metrics"]["cost"]/cases["revised"]["metrics"]["cost"]-1)

```

```

462         fixed_plan_deficit_kwh=float(np.maximum(gap,0).sum()),
463         violations=result["metrics"]["violations"])
464     noise.append(row); print("噪声检验", row, flush=True)
465     final = cases["revised"]["trajectory"] # 为题目指定时段另外输出便于核对的汇总。
466     specified = [{"start_hour": t/6, "purchase_kwh": float(final["g"][t])}
467                 for t in (60, 72, 84, 96, 108, 120)]
468     blocks = [{"start_hour": 4*j, "end_hour": 4*(j+1),
469              "charge_kwh": float(final["c"][24*j:24*(j+1)].sum()*DT),
470              "discharge_kwh": float(final["d"][24*j:24*(j+1)].sum()*DT)} for j in range(6)]
471     summary = dict(environment=dict(python=sys.version, numpy=np.__version__, scipy=scipy.
472                                   __version__),
473                    no_storage_cost=float(data[:,0]@np.maximum(data[:,1]-data[:,2],0)*DT),
474                    load_kwh=float(data[:,1].sum()*DT), pv_kwh=float(data[:,2].sum()*DT),
475                    cases=sensitivity, noise=noise, specified=specified, storage_blocks=blocks,
476                    initial_kwh=float(final["E"][0]), final_kwh=float(final["E"][-1]))
477     (out/"summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf
478                                     -8")
479     if len(sweep)>1:
480         draw(out, data, cases, sweep)
481     print("完成: 144段计划、各层求解状态及检验结果位于", out.resolve(), flush=True)
482
483 if __name__ == "__main__":
484     try:
485         main()
486     except (ValueError, RuntimeError, OSError, ImportError) as exc:
487         print(f"运行终止: {exc}", file=sys.stderr)
488         sys.exit(1) # 不输出虚构的成功结论。

```